



AsterVoid:

Born from the void,
made for the stars

THINK IN LIGHT , CODE IN VOID.

Table of contents

Language
Specification

01

03

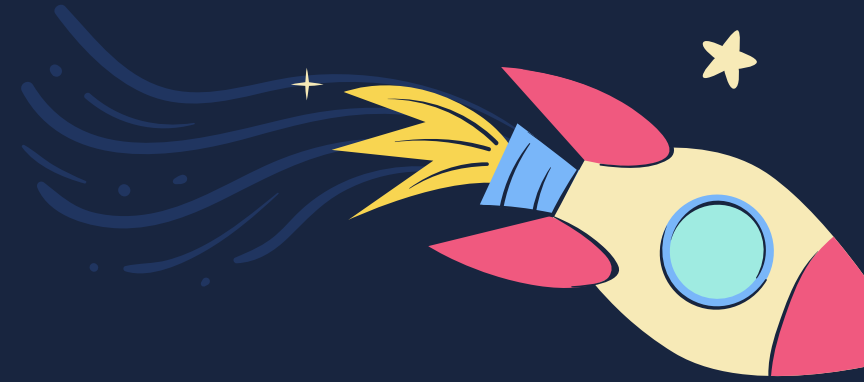
Parser

Scanner

02

04

Semantic Analysis



The background is a dark blue space scene. It features several stylized celestial bodies: a pink planet with a face in the top left, a yellow comet with a face in the top center, a yellow ringed planet with a face in the bottom right, and a blue satellite in the bottom left. There are also various constellations represented by white dots and lines, and numerous small white stars. Concentric blue circles represent orbits.

Introduction

AsterVoid is a space-themed,
C++-style programming language
designed for clarity, teaching, and
playful expression

01

Language Specifecation



Language Specifecation

Language Features

Import

#launch <bigbang>

I/O:

- shine → output/print
- moon → input/read

Control Flow

- phase → if
- eclipse → else
- orbit → while
- rotate → for
- supernova → switch

Keywords

- mass → int
- flux → float
- quantum → double
- star → string
- nebula → char
- truth → bool
- vacuum → void

Complete Code Example:



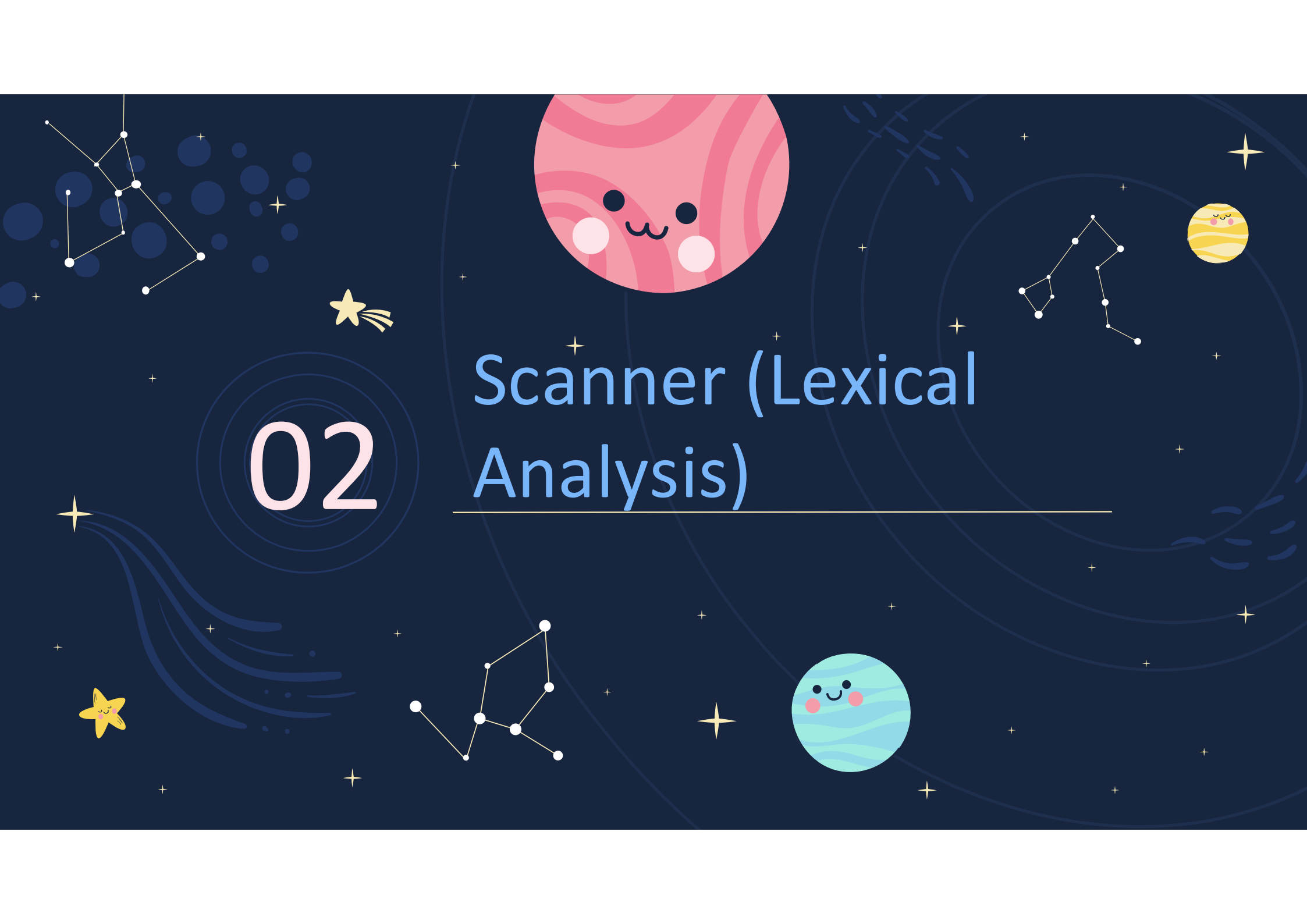
```
1  ** Welcome to AsterVoid IDE!
2  #launch <bigbang>
3  mass calculatePower(mass base, mass exponent){
4      mass result = 1;
5      rotate (mass i = 0; i < exponent; i = i + 1) {
6          result = result * base;
7      }
8      blackHole result;
9  }
10 vacuum milkyway() {
11     mass number;
12     flux temperature = 36.5;
13     shine("Enter a number: ");
14     moon(number);
15     phase (number > 10) {
16         shine("Large number!");
17         mass power = calculatePower(number, 2);
18         shine("Square is: ", power); }
19     eclipse { shine("Small number!");
20 }
21 }
```



Astervoid

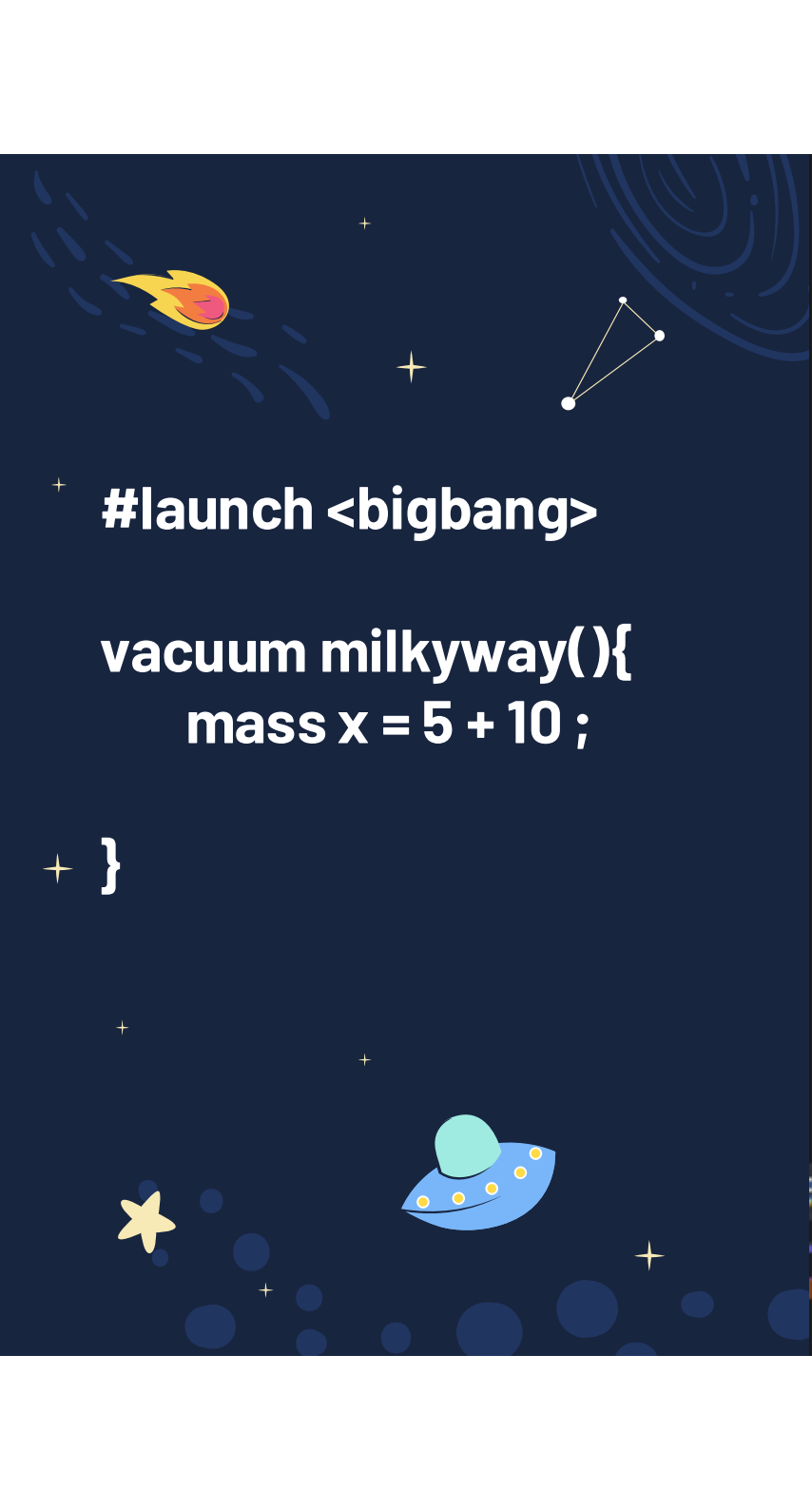
Think in light. Code in void.
Precision forged in the dark



The background is a dark blue space-themed illustration. It features several constellations represented by white dots connected by thin white lines. There are numerous small white stars and larger, stylized planets. One planet is pink with horizontal stripes and a smiling face with two black eyes and a wavy mouth. Another is yellow with horizontal stripes and a smiling face. A third is light blue with horizontal stripes and a smiling face. There are also some blue comet-like streaks and a yellow star with a face. The overall style is cute and whimsical.

02

Scanner (Lexical Analysis)



#launch <bigbang>

**vacuum milkyway(){
 mass x = 5 + 10 ;**

}

=== LEXICAL ANALYSIS ===

TOKENS:

#launch <bigbang> -----> (LAUNCH)

vacuum -----> (VACUUM)

milkyway -----> (MILKYWAY)

(-----> (LEFT_PAREN)

) -----> (RIGHT_PAREN)

{ -----> (LEFT_BRACE)

mass -----> (MASS)

x -----> (IDENTIFIER)

= -----> (EQUAL)

5 -----> (NUMBER)

+ -----> (PLUS)

10 -----> (NUMBER)

; -----> (SEMICOLON)

} -----> (RIGHT_BRACE)

-----> (END_OF_FILE)

Code snippet:

```
#include "Scanner.h"
#include <unordered_map>
#include <iostream>

Scanner::Scanner(const std::string& sourceCode) : source(sourceCode) {}
```

```
▼ std::vector<Token> Scanner::scanTokens() {
    while(!isAtEnd()) {
        skipWhitespace();
        if (isAtEnd()) break;
        start = current;
        startCol = col;
        Token token = scanToken();
        tokens.push_back(token);
    }
    tokens.push_back(makeToken(TokenType::END_OF_FILE, "", ""));
    return tokens;
}
```

```
▼ Token Scanner::scanToken() {
    char ch = advance();

    switch(ch) {
        // SINGLE CHAR
        case '(': return makeToken(TokenType::LEFT_PAREN, "(");
        case ')': return makeToken(TokenType::RIGHT_PAREN, ")");
        case '{': return makeToken(TokenType::LEFT_BRACE, "{");
        case '}': return makeToken(TokenType::RIGHT_BRACE, "}");
        case ',': return makeToken(TokenType::COMMA, ",");
        case '.':
            if(isDigit(peek())) return number();
            return makeToken(TokenType::DOT, ".");
        case ';': return makeToken(TokenType::SEMICOLON, ";");
        case ':': return makeToken(TokenType::COLON, ":");
        case '#': return directive();
    }
}
```

03

Parser (Syntax Analysis)



Snippet of AsterVoid Grammer:

Program $\rightarrow$ LaunchDirective MainFunction EOF

MainFunction $\rightarrow$ vacuum milkyway () { StatementList }

LaunchDirective $\rightarrow$ LAUNCH

StatementList $\rightarrow$ Statement StatementList | ϵ

Declaration $\rightarrow$ Type Identifier DeclarationTail ; DeclarationTail $\rightarrow$ = Expr | ϵ

Type $\rightarrow$ mass | flux | quantum | nebula | star | truth | vacuum

Assignment $\rightarrow$ Identifier = Expr ;



#launch <bigbang>

**vacuum milkyway(){
mass x = 5 + 10 ;**

}

=== SYNTAX ANALYSIS ===
Syntax OK!

Parse Tree:

PROGRAM

LAUNCH_DIRECTIVE <#launch <bigbang>>

GLOBAL_STATEMENT_LIST

MAIN_FUNCTION

TOKEN <vacuum>

TOKEN <milkyway>

TOKEN <(>

TOKEN <)>

TOKEN <{>

STATEMENT_LIST

DECLARATION

TOKEN [mass] <mass>

TOKEN [x] <x>

DECLARATION_TAIL

TOKEN [=] <=>

EXPRESSION

TERM

FACTOR [5] <5>

TOKEN [+] <+>

TERM

FACTOR [10] <10>

TOKEN <;>

TOKEN <}>

Code snippet:

```
// ----- Statements -----  
✓ std::shared_ptr<Node> Parser::declaration() {  
    auto declNode = std::make_shared<Node>(NodeType::DECLARATION);  
  
    // Type  
    Token typeToken = peek();  
    advance();  
    declNode->addChild(std::make_shared<Node>(NodeType::TOKEN, typeToken, typeToken.lexeme));  
  
    Token idToken = peek();  
    consume(TokenType::IDENTIFIER, "Expected identifier after type");  
    declNode->addChild(std::make_shared<Node>(NodeType::TOKEN, idToken, idToken.lexeme));  
  
    auto tailNode = declarationTail();  
    if (tailNode) declNode->addChild(tailNode);  
  
    Token semiToken = peek();  
    consume(TokenType::SEMICOLON, "Expected ';' after declaration");  
    declNode->addChild(std::make_shared<Node>(NodeType::TOKEN, semiToken));  
  
    return declNode;  
}
```



04

Symbol Table:



```
#include "SymbolTable.h"

// ===== Symbol Implementation =====
Symbol::Symbol() : name(""), type(""), line(0), col(0), scope(0) {}

Symbol::Symbol(const std::string& n, const std::string& t, int l, int c, int s)
    : name(n), type(t), line(l), col(c), scope(s) {}

// ===== SymbolTable Implementation =====
SymbolTable::SymbolTable() : currentScope(0) {
    // Initialize with global scope
    scopes.push_back(std::unordered_map<std::string, Symbol>());
}

void SymbolTable::pushScope() {
    currentScope++;
    scopes.push_back(std::unordered_map<std::string, Symbol>());
}

void SymbolTable::popScope() {
    if (scopes.size() > 1) { // Never remove global scope
        scopes.pop_back();
        currentScope--;
    }
}
```

```
1 #launch <bigbang>
2
3 vacuum milkyway() {
4     mass x = "Compiler" ;
5     mass x ;
6     shine(y);
7     phase(t > 1){
8         mass l = 10 ;
9     }
10    shine(l);
11 }
12 |
```

=== SEMANTIC ANALYSIS ===

Semantic Error: 'x' redeclared at line 5, col 11

Semantic Error: Undefined variable 'y' at line 6, col 12

Semantic Error: Undefined variable 't' at line 7, col 12

Semantic Error: Undefined variable 'l' at line 10, col 12

Semantic Errors Found:

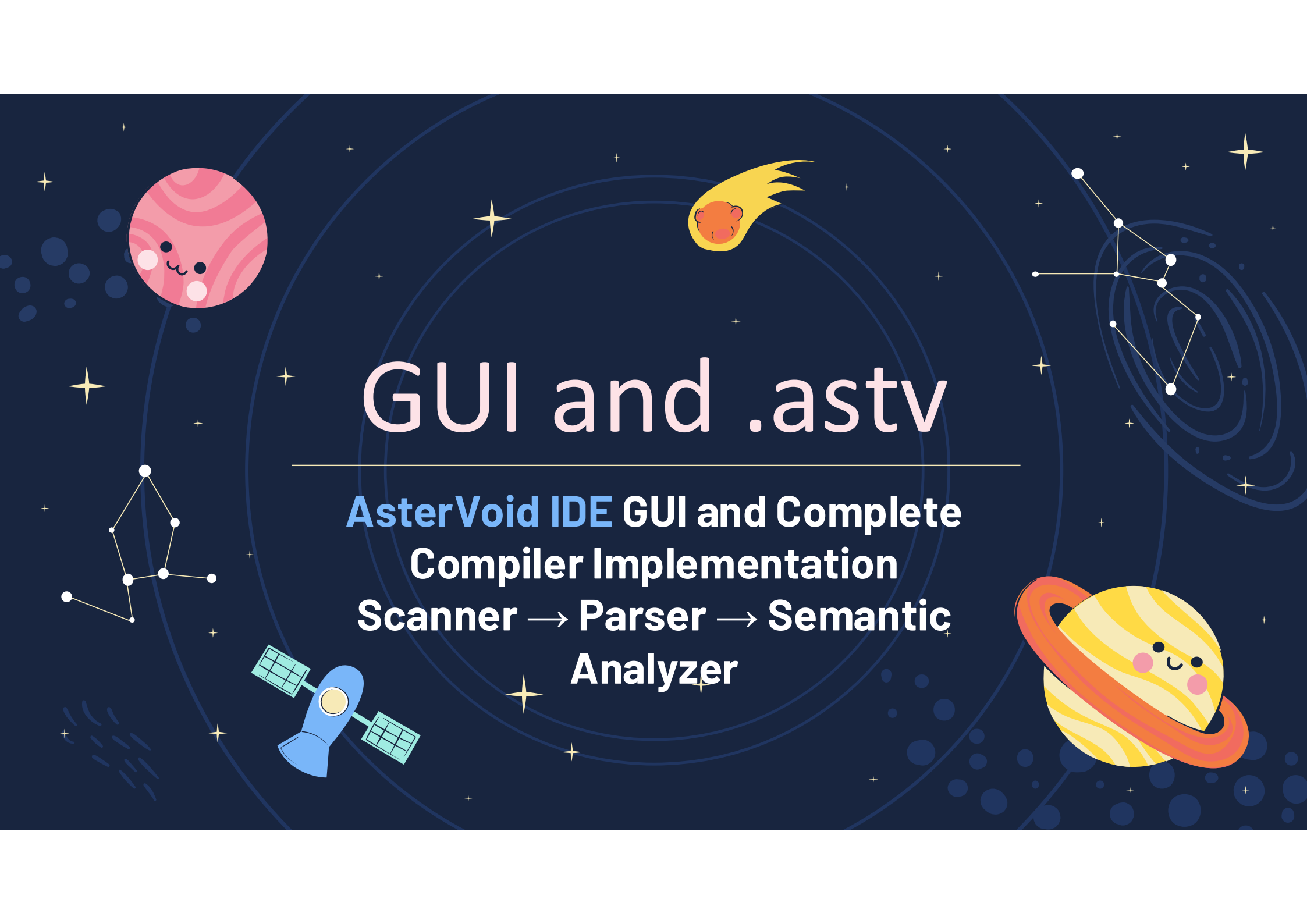
Type mismatch in declaration of 'x' at line 4: cannot assign star to mass

Type mismatch in condition: cannot compare error with mass

Code snippet:

```
void SemanticAnalyzer::analyzeIfStatement(std::shared_ptr<Node> node) {  
    // Find and analyze condition  
    for (size_t i = 0; i < node->getChildCount(); i++) {  
        auto child = node->getChild(i);  
        if (child->type == NodeType::CONDITION) {  
            std::string condType = analyzeCondition(child);  
            if (condType != "bool") {  
                reportError("Phase condition must be boolean type",  
                    child->token.line, child->token.col);  
            }  
        }  
        else if (child->type == NodeType::STATEMENT_LIST) {  
            symbolTable.pushScope();  
            analyzeStatementList(child);  
            symbolTable.popScope();  
        }  
    }  
}
```



The background is a dark blue space scene. It features several stylized celestial bodies: a pink planet with a face in the top left, a yellow comet with a face in the top center, a yellow ringed planet with a face in the bottom right, and a blue satellite in the bottom left. There are also various star patterns, including simple four-pointed stars and constellations of dots connected by lines. Concentric blue circles represent orbits.

GUI and .astv

**AsterVoid IDE GUI and Complete
Compiler Implementation
Scanner → Parser → Semantic
Analyzer**



"born from the void, made for the stars"

Project Files

- milkyway.astv
- planet.astv
- orbit.astv
- examples/

New Open Save Launch

```
1  ** Welcome to AsterVoid IDE!  
2  #launch <bigbang>  
3  
4  vacuum milkyway() {  
5      shine("Hello, Galaxy!\n");  
6  
7      mass count = 0;  
8      orbit (count < 5) {  
9          shine("Star ", count, "\n");  
10         count = count + 1 ;  
11     }  
12  
13     blackHole 0;  
14 }  
15 ** hello
```

⚡ OUTPUT TERMINAL

Ready to launch...

File Run

**AsterVoid**

"Born from the void, made for the stars"

Project Files

● milkyway.astv

🔥 planet.astv

🌙 orbit.astv

📁 examples/

New

Open

Save

Launch

```
1  ** Welcome to AsterVoid IDE!
2  #launch <bigbang>
3
4  vacuum milkyway() {
5      shine("Hello, Galaxy!\n");
6      mass count = 0;
7      |
8      blackHole;
9  }
```

⚡ OUTPUT TERMINAL

Ready to launch...

Ready to launch

AsterVoid (.astv) | Line: 7, Col: 5



"Born from the void, made for the stars"

Project Files

- milkyway.astv
- planet.astv
- orbit.astv
- examples/

New Open Save Launch

```

✈ Launching AsterVoid program...

📄 Code saved to: C:/temp/temp_code.astv

✍ Using compiler: C:/Users/Fix-Dell/source/repos/AsterVoid/x64/Debug/AsterVoid.exe

📁 Working directory: C:/temp

⌚ Running...
```

Compiling file: temp_code.astv

```

=== LEXICAL ANALYSIS ===
TOKENS:
#launch <bigbang> -----> (LAUNCH)
vacuum -----> (VACUUM)
milkyway -----> (MILKYWAY)
( -----> (LEFT_PAREN)
) -----> (RIGHT_PAREN)
{ -----> (LEFT_BRACE)
shine -----> (SHINE)
( -----> (LEFT_PAREN)
Hello, Galaxy!\n -----> (STAR)
) -----> (RIGHT_PAREN)
; -----> (SEMICOLON)
mass -----> (MASS)
count -----> (IDENTIFIER)
= -----> (EQUAL)
0 -----> (NUMBER)
; -----> (SEMICOLON)
blackHole -----> (BLACKHOLE)
; -----> (SEMICOLON)
} -----> (RIGHT_BRACE)
-----> (END_OF_FILE)
```

```

=== SYNTAX ANALYSIS ===
Syntax OK!
```

```

Parse Tree:
PROGRAM
```

Compiling...

AsterVoid (.astv) | Line: 7, Col: 5



AsterVoid

"Born from the void, made for the stars"

Project Files

milkyway.astv

planet.astv

orbit.astv

examples/

New Open Save Launch

```
Parse Tree:
PROGRAM
  LAUNCH_DIRECTIVE <#launch <bigbang>>
  GLOBAL_STATEMENT_LIST
  MAIN_FUNCTION
    TOKEN <vacuum>
    TOKEN <milkyway>
    TOKEN <(>
    TOKEN <(>
    TOKEN <(>
    STATEMENT_LIST
      OUTPUT_STATEMENT
        TOKEN <shine>
        TOKEN <(>
        EXPRESSION_LIST
          EXPRESSION
            TERM
              FACTOR [Hello, Galaxy!\n] <Hello, Galaxy!\n>
            TOKEN <(>
            TOKEN <;>
          DECLARATION
            TOKEN [mass] <mass>
            TOKEN [count] <count>
          DECLARATION_TAIL
            TOKEN [=] <=>
            EXPRESSION
              TERM
                FACTOR [0] <0>
            TOKEN <;>
          RETURN_STATEMENT
            TOKEN <blackHole>
            TOKEN <;>
          TOKEN <(>

```

=== SEMANTIC ANALYSIS ===

Semantic analysis passed!

=== COMPILATION SUCCESSFUL ===

✓ Program completed successfully!

*** Thank you ***

هنا صلاح مسعود
سما عصام جلال
ميّار محمد أحمد